Table 1-7. *(continued)*

`is_writable` `is_writeable()`	Test file exists and is writ(e)able
`mkdir()`	Create a directory
`rename()`	Rename a file or directory
`rmdir()`	Delete an empty directory
`symlink()`	Create a symbolic link
`touch()`	Update access and modification times of a file will create an empty file if it doesn't exist already

If you have any familiarity at all with *nix filesystem commands, the functions above will look very familiar, which is to be expected because they are just wrappers around the system calls. Mileage may vary on Windows systems when it comes to symbolic links, which are handled very differently compared to Linux and Mac.

Remote Files and Streams

`fopen()` is not just limited to the local filesystem. By adding some more information to the path string, it is possible to access data from remote systems. Of course, it isn't possible to determine just how much data might be sent back by the other end, so I'll introduce another function: `feof()` tests a file handle to see if it is in an "EOF" (end of file) state. For local files this is simple: the file size is known, so when all bytes have been read, the handle can be set to the EOF state. For remote data, the underlying network socket has EOF set when the socket closes (remember, *nix systems abstract network data streams to the filesystem; this is what allows you to handle them like any other file):

```
$handle = fopen('https://github.com', 'r');

while (!feof($handle)) {
    print fgets($handle);
}

var_dump($handle);
fclose($handle);
var_dump($handle);
```